

Unit – I

Software Process Models

By: Yagnik Hathaliya

Major Learning Outcomes

- **Software**
 - *Definition, Characteristics*
- **Software Application Domain**
 - *System Software, Application Software, Embedded Software, Web Application, AI Software*
- **Software Engineering**
 - *Definition, Layered Technology Approach*
- **Generic Framework Activities**
- **Umbrella Activities**
- **Software development**
- **Software Development Models**
 - *Waterfall Model*
 - *Incremental Process Model*
 - *Prototyping Model*
 - *Spiral Model*
- **Agile Development Model**
 - *Agile Principles*
 - *Agile Model vs Waterfall Model*
- **Types of Widely Used Agile Models**
 - *Extreme Programming (XP)*
 - *Scrum*

Software (IEEE Definition)

- Software is a set of instructions, data, or programs used to operate computers and execute specific tasks.
- IEEE Defines: Software is the “Collection of computer programs, procedures, rules, associated documents and concerned data with the operation of data processing system.”

Software Application Domain

- **System Software**

- Enables hardware and application software to work together, it is responsible for controlling and integrating the hardware components of a system so that the software and the users can work with them.
- Example: Operating system, Device Drivers etc.

- **Application Software**

- it is used to accomplish some specific task and It may be a collection of small programs such as word processing, web browsing, or gaming.
- Example: Microsoft Word, Microsoft Excel, Google Chrome etc.

- **Embedded Software**

- Embedded Software is specialized software designed to control and operate devices or machines that are not typically thought of as computers.
- It is embedded within hardware to perform dedicated tasks, often in real-time environments.
- Example: Smartwatches, Fitness Trackers, Home Automation Systems, Air Conditioning Systems, Medical Devices etc.

- **Web Application Software**

- Web Application Software refers to programs or applications that run on web servers and are accessed through a web browser over the internet.
- Example: e-Commerce Sites, Social Media Sites, Banking Facility etc.

- **Artificial Intelligence Software**

- Artificial Intelligence (AI) Software refers to programs designed to simulate human intelligence processes, such as learning, reasoning, problem-solving, and decision-making.
- Example: Google Assistant , ChatGPT , Amazon Alexa etc.

Program Vs Software

Program	Software
Small in Size	Very Large in Size
Developed by individual	Developed by Large number of Developers
Limited functionality	Multiple functionality
Little documentation	Large documentation
User interface may not very important	User interface may very important
Develop using programmer's individual style	Develop using Software Engg. principle

Characteristics of a Software

- These are the attributes reflect the quality of a software product.
- Following are the characteristics of Good Software.
- **Operational Characteristics:**
 1. **Correctness:** The software which we are making should meet all the specifications stated by the customer.
 2. **Usability:** The amount of efforts or time required to learn how to use the software should be less, This makes the software user-friendly.
 3. **Integrity :** quality software should not have side effects.
 4. **Reliability :** The software product should not have any defects. Not only this, it shouldn't fail while execution.
 5. **Efficiency :** The software should make effective use of the storage space and execute command as per desired timing requirements.
 6. **Security :** The software shouldn't have side effects on data / hardware.
 7. **Safety:** The software should safe.

- **Revision Characteristics:**

1. **Maintainability** : Maintenance of the software should be easy for any kind of user.
2. **Flexibility** : Changes in the software should be easy to make.
3. **Extensibility** : It should be easy to increase the functions performed by it.
4. **Scalability** : It should be very easy to upgrade it for more work.
5. **Testability** : Testing the software should be easy.
6. **Modularity** : Software must be divided in to separate individual working modules or parts

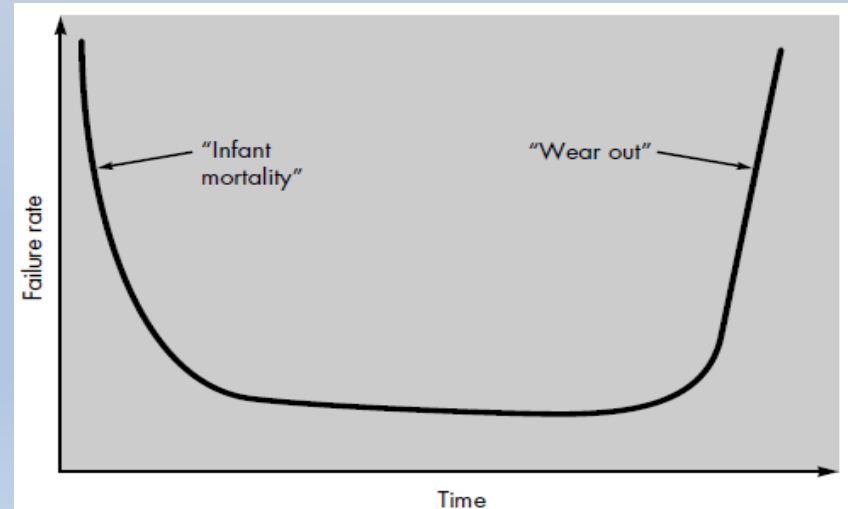
- **Transition Characteristics:**

1. **Interoperability** : Interoperability is the ability of software to exchange information with other applications and make use of information transparently.
2. **Reusability** : If we are able to use the software code with some modifications for different purpose then we call software to be reusable.
3. **Portability** : The ability of software to perform same functions across all environments and platforms, demonstrate its portability.

Some Special Characteristics:

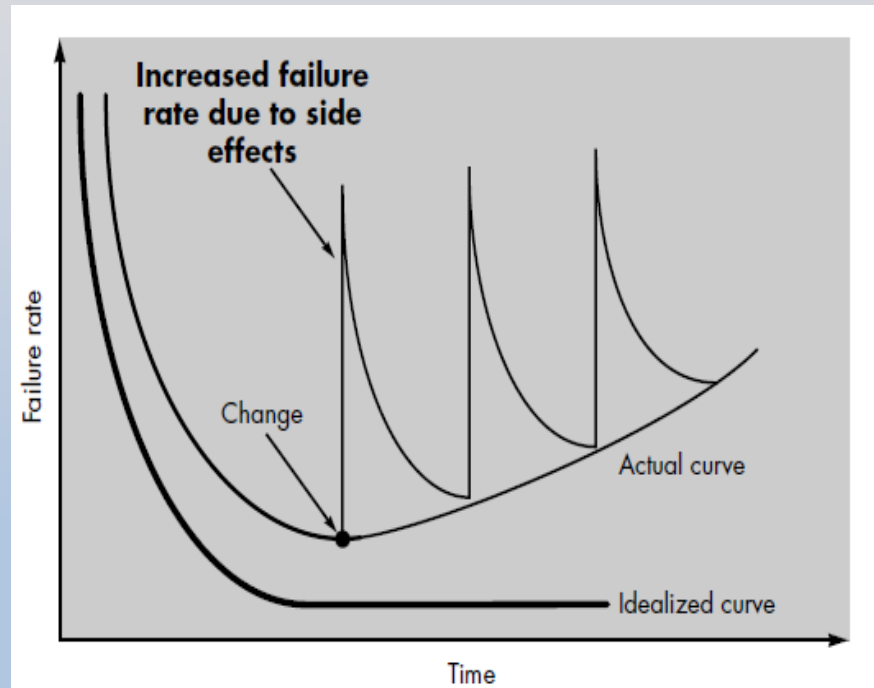
1. Software Doesn't Wear out.

- Hardware can damage after running time, It can be affected by environmental effects, so, the failure rate rises.
- “bathtub curve” shows hardware failure, there are three phases in h/w life, initially failure rate is much more, but after testing and defects are corrected, failure rate come down.
- In it, h/w is much more useful and chance of failure is quite low.
- As time passes, however, the failure rate rises again as hardware components suffer from the affects of dust, vibration, abuse, temperature extremes, and many other environmental factors.
- So simply, hardware does wear out.



H/W failure curve

- Software is not highly affected by environmental effects. The “idealized curve” shows software failure.
- In the early stage, due to lot many errors, software could have high failure. But it becomes reliable as time passes instead of wearing out. Software become reliable.
- Software may be retired due to new requirements, new expectations etc.
- Hence, software doesn't wear out, but it may be got worse.



S/W failure curve

2. S/W is Engineered, Not Manufactured.

- Once a product is manufactured, it is not easy to modify it, While in case of software we can easily change or modify It for later use, even making multiple copies of software is a very easy.
- In hardware, costing is due to assembly of raw material and other processing expenses while in software development no assembly needed like hardware. Hence, software is not manufactured as it is developed or it is engineered.

3. Software is Flexible for Custom Built

- A software program or product can be built on user requirements basis or custom built.

4. Software gives Reusability of Components

- All large software divided in to some modules or components, this module developed individually and than integrated to develop a complete software.
- All these module can be used for other similar types of software
- So, Software gives or provide reusability of components.

Software Engineering (SE) Definition

- Software engineering is an engineering approach for software development.
- A small program can be written without SE principles, But to develop large software with good quality and cost effective we have to use SE principles.
- SE is an engineering discipline that covers all aspects of software from specification to maintenance.
- Three main aspects of SE is :(Quality software at agreed cost in schedule time)
 1. Provide quality product
 2. Expected cost
 3. Complete work on agreed schedule
- IEEE Definition of SE:
 - Software Engineering is the application of a symmetric , disciplined and quantifiable approach to the development, operation and maintenance of software.

- **Need (Necessity) of Software Engineering:**
 - To help developers to obtain high quality software product.
 - To develop the product in appropriate manner using life cycle models.
 - To acquire skills to develop large programs.
 - To acquire skills to be a better programmer.
 - To provide a software product in a timely manner.
 - To provide a software product at an agreed cost.
 - To develop ability to solve complex programming problems.
 - Also learn techniques of: specification, design, user interface development, testing, project management, etc.

Software Engineering Layered Technology Approach

- The Layered Technology Approach in Software Engineering is a structured framework for designing, developing, and maintaining software. It organizes software engineering activities into layers, with each layer building upon and relying on the functionalities of the layer below it.
- This approach promotes modularity, systematic execution, and a clear understanding of processes.
- Four Layers are there:
 1. Quality Focus Layer
 2. Process Layer
 3. Method Layer
 4. Tools Layer

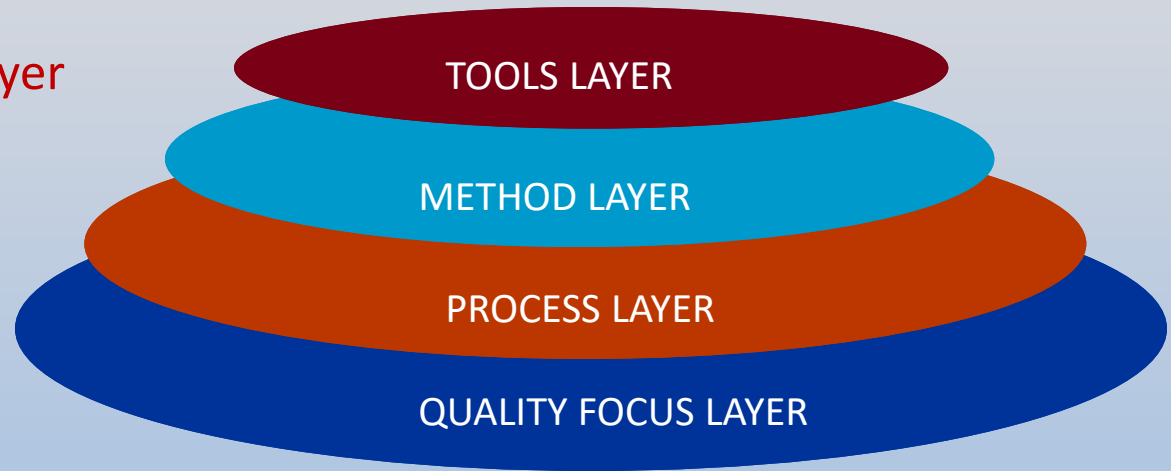


Fig: SE Layers Technology Approach

- **Quality Focus Layer**

- The foundation of the layered approach, ensuring that quality is maintained across all aspects of software development, It checks whether the output meets with its requirement specifications or not.
- Key Task: Quality management processes.

- **Process Layer**

- Defines the framework and workflows for planning, managing, and monitoring the software engineering activities.
- Main idea is to deliver software in a timely manner with quality to satisfy users, It also deals with technical and management issues of software development.
- Key Task: Specifies “how” to do things and Guides the execution of tasks in a disciplined manner.

- **Method Layer**

- Contains the technical methods and procedures for creating and building software.
- It includes different tasks such as user communication, requirement analysis, coding, testing, maintenance etc.
- Key Task: Techniques for analysis, design, coding, testing, and maintenance.

- **Tools Layer**

- Provides different types of tools for software development to execute process in proper manner.
- Key Task: Tools for code generation, testing, debugging, and project management.

- **Advantages**

- Systematic Workflow: Provides a clear hierarchy and flow of activities.
- Modularity: Promotes separation of concerns, making it easier to manage complexities
- Scalability: Adaptable to small and large-scale projects.
- Quality Assurance: Ensures that all activities contribute to maintaining high-quality outcomes.

Generic Framework Activities (Software Process)

- Generic Framework Activities are the core activities common to all software engineering processes, regardless of the specific methodology or model used. These activities provide a structured approach to software development and are adaptable to the unique needs of each project.
- In SE there are six generic framework activities that is explained below,
 1. **Communication**
 - Establish clear understanding between stakeholders and developers.
 2. **Planning**
 - Define the roadmap for the project, including timelines, resources, and tasks.
 3. **Modeling**
 - In this a model will be created to better understand the requirements.
 4. **Construction**
 - In this code will be generated and tested.
 5. **Deployment**
 - In this complete version of the software is represented to the customers to evaluate and they give feedback based on evaluation.
 6. **Maintenance**
 - Ongoing process of fixing bugs, making enhancements, and improving the performance of the software system

Umbrella Activities (Software Process)

- Umbrella Activities in software engineering are primary tasks that span the entire software development lifecycle, supporting the core framework activities, these activities ensure quality, manage risks, and maintain consistency across all phases of the project. They are not confined to a single stage but are performed continuously throughout the development process.
- In SE there are Eight Umbrella Activities that are listed below,
 1. **Project Tracking and Control**
 - Monitor project progress and ensure it stays on track concerning timelines, resources, and objectives.
 2. **Risk Management**
 - Identify, analyze, and moderate risks that could impact project success.
 3. **Software Quality Assurance**
 - Ensure the software meets predefined quality standards and user requirements.
 4. **Formal Technical Review**
 - Detect defects early and ensure technical accuracy of deliverables.
 5. **Measurement**
 - Collect and analyze data to evaluate project performance and software quality.

6. Software Configuration Management

- Manage and control changes to software objects throughout the project.

7. Reusability Management

- Promote reuse of software components to decrease development time and costs.

8. Work Product Preparation and Production

- Prepare and manage deliverables such as documentation, reports, and software artifacts.

Software Development Life Cycle (SDLC)

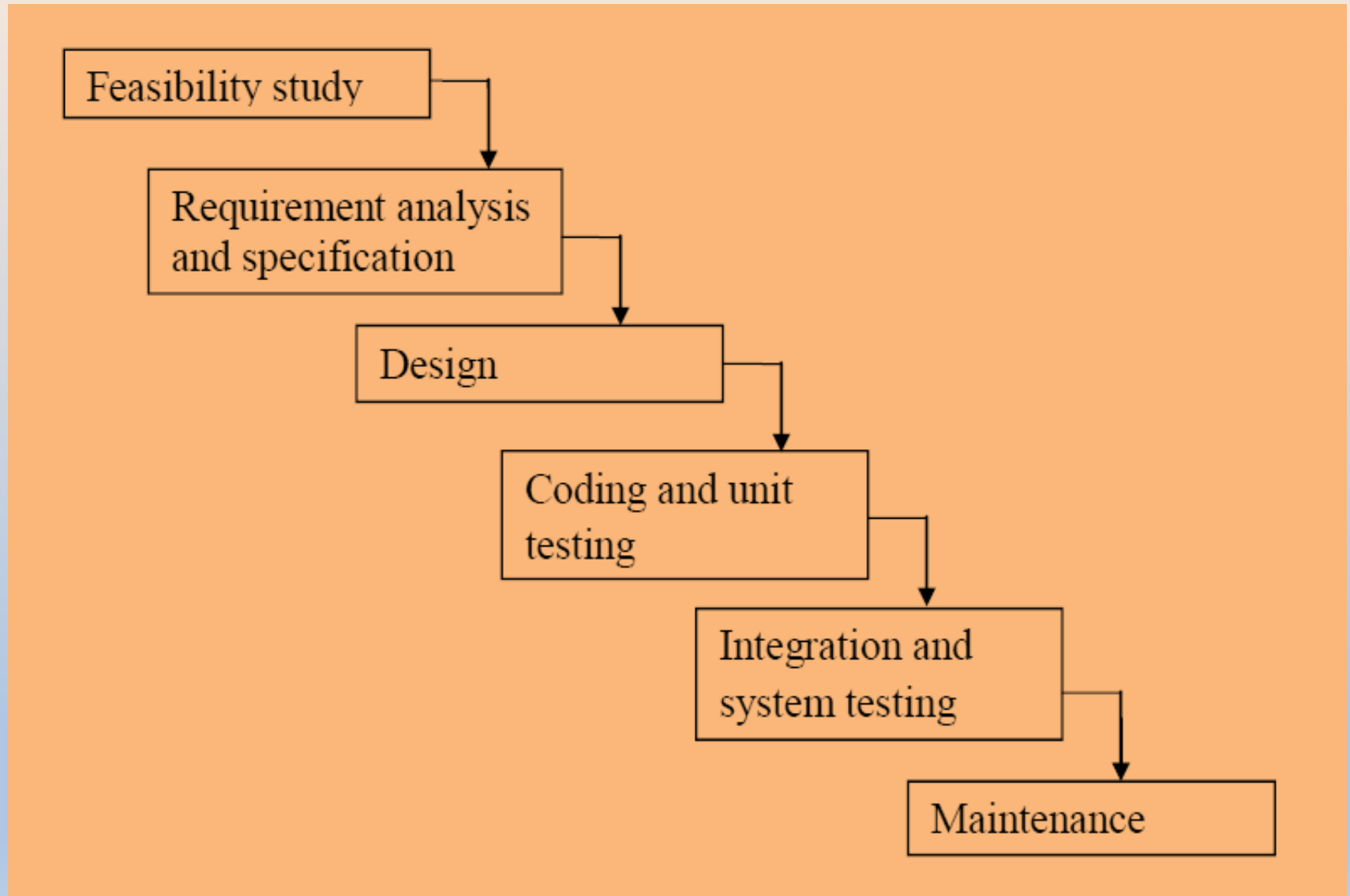
- The Software Development Life Cycle (SDLC) is a structured process for developing software applications, ensuring they are high-quality, efficient, and meet user requirements.
- It encompasses a series of phases that provide a framework for planning, building, testing, deploying, and maintaining software.
- General phases of software development life cycle are given below,
 1. **Planning** (Define the project's scope, goals, and feasibility)
 2. **Requirement Analysis** (Understand and document the functional and non-functional requirements of the software)
 3. **Design** (Create a blueprint for the system that addresses requirements)
 4. **Implementation (Coding)** (Develop the software based on the design specifications)
 5. **Testing** (Verify that the software functions as intended and is free of defects)
 6. **Deployment** (Deliver the software to the end-users and make it operational in its environment)
 7. **Maintenance** (ensure the software continues to meet user needs after deployment)

- **Advantages of Software Development Life Cycle Models:**
 - Structured Workflow
 - Improved Planning and Management
 - Requirement Clarity
 - Quality Assurance
 - Risk Management
 - Efficient Resource Utilization
 - Enhanced Collaboration and Communication
 - Timely Delivery
 - Focus on Customer Satisfaction

Classical Waterfall Model

- This model was originally proposed by Royce (1970).
- The classical waterfall model is a sequential software development methodology that emphasizes a linear and structured approach to project management.
- It was one of the first process models introduced in software engineering and is best suited for projects with clearly defined requirements and predictable outcomes.
- It is also called “**traditional waterfall model**” or “**conventional waterfall model**”.
- Phases of this model are:
 1. **Feasibility Study**
 2. **Requirements Analysis and Specification**
 3. **Design**
 4. **Coding and Unit Testing**
 5. **Integration and System Testing**
 6. **Maintenance**

- The Classical Waterfall Model is depicted below,



- Feasibility Study

- The main aim of feasibility study is to determine whether it would be financially and technically feasible to develop the product.
- At first project managers or team leaders try to have a rough understanding of what is required to be done by visiting client side.
- They study different input data and output data and processing require on these data.
- After overall understanding of the problem, they investigate the different solutions that are possible.
- Then Examine each solution in terms of Resources, Cost and Time.
- Based on this analysis they pick the best solution and determine whether the solution is feasible financially and technically.
- They check whether the customer budget would meet the cost of the product and whether they have sufficient technical expert in the area of development.

- Requirements Analysis and Specification

- The aim of the requirements analysis and specification phase is to understand the exact requirements of the customer and to document them properly.
- This phase consists of two distinct activities, namely
 - Requirements Gathering and Analysis, and
 - Requirements Specification
- The goal of the requirement's gathering activity is to collect all relevant information from the customer regarding the product and clearly understand the customer requirements so that incompleteness and inconsistencies are removed.
- The requirements analysis activity is begun by collecting all relevant data regarding the product to be developed from the users of the product and from the customer through interviews and discussions.
- The data collected from such a group of users usually contain several contradictions and ambiguities, since each user typically has only a partial and incomplete view of the system.
- Therefore, it is necessary to identify all ambiguities and contradictions in the requirements and resolve them through further discussions with the customer.
- After all ambiguities, inconsistencies, and incompleteness have been resolved and all the requirements properly understood, the requirements specification activity can start.
- During this activity, the user requirements are systematically organized into a Software Requirements Specification (SRS) document.

- Design

- The goal of the design phase is to transform the requirements specified in the SRS document into a structure that is suitable for implementation in some programming language, during the design phase the software architecture is derived from the SRS document.
- Two different approaches are available for design :
 - **Traditional Design Approach**
 - In this first structure analysis is performed where the structure of the problem is examined and then structure design is performed.
 - **Object-Oriented Design Approach**
 - In this first Different object identified and then relationship among these objects are identified and then detailed design is performed.

- Coding and Unit Testing

- The purpose of the coding and unit testing phase (sometimes called the implementation phase) of software development is to translate the software design into source code.
- Each component of the design is implemented as a program module.
- The end-product of this phase is a set of program modules that have been individually tested.
- During this phase, each module is unit tested to determine the correct working of all the individual modules.

- Integration and System Testing

- During the integration and system testing phase, the modules are integrated in a planned manner.
- The different modules are almost never integrated in one shot, Integration is normally carried out incrementally over a number of steps, during each integration step, the partially integrated system is tested and a set of previously planned modules are added to it.
- Finally, when all the modules have been successfully integrated and tested, system testing is carried out.
- The goal of system testing is to ensure that the developed system conforms to its requirements laid out in the SRS document.
- System Testing usually consists of three different kinds of testing activities:
 1. **α – Testing**: system testing performed by the development team.
 2. **β – Testing**: system testing performed by a friendly set of customers.
 3. **Acceptance Testing**: system testing performed by the customer himself after the product delivery to determine whether to accept or reject the delivered product.

- Maintenance

- Maintenance are carried out to address any issues or changes that arise and it requires much more than the effort necessary to develop the product itself.
- Maintenance involves performing any one or more of the following three kinds of activities
 - Correcting errors that were not discovered during the product development phase, this is called **Corrective Maintenance**.
 - Improving the implementation of the system, and enhancing the functionalities of the system according to the customer's requirements, this is called **Perfective Maintenance**.
 - Porting the software to work in a new environment is called **Adaptive Maintenance**.

- Advantages

- Simple Model
- Easy to Understand
- Easy to Use
- Each phase has well defined input and outputs
- Works well for smaller projects
- It divides complex tasks into smaller, more manageable works.

- Disadvantages

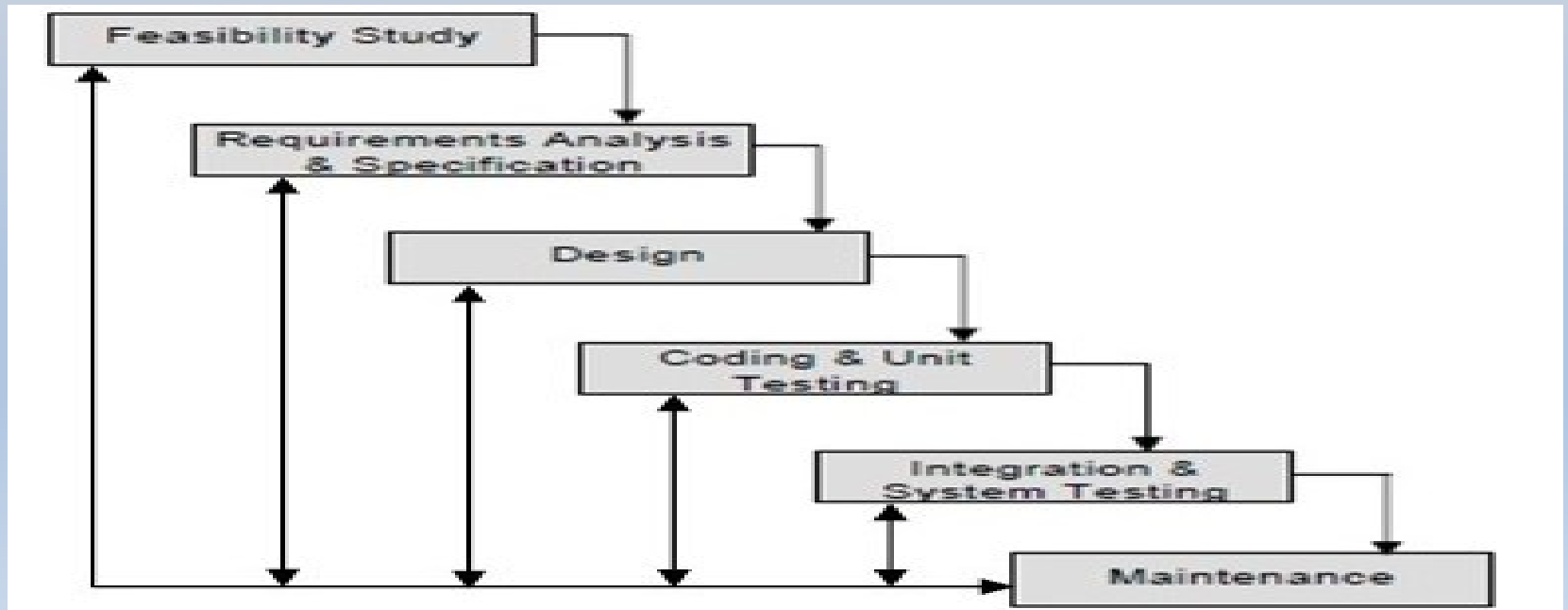
- It is a theoretical model.
- It may happen that the error may be generated at any phase and encountered in later phase. So, it's not possible to go back and solve the error in this model.
- High amounts of risk and uncertainty.

- Applications

- This model is used only when the requirements are very well known, clear and fixed.
- When the Product is stable or Project is short

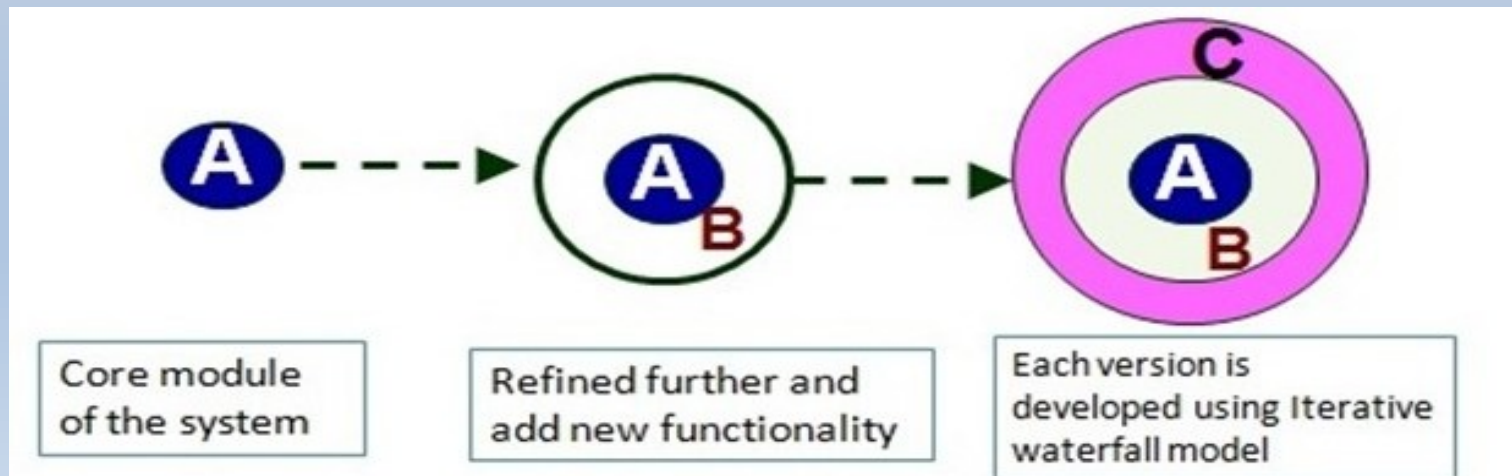
- Iterative Waterfall Model

- Classical Waterfall model is idealistic because, it assumes that no defect is introduced during any development activity, but in practice defects do get introduced in almost every phase of the life cycle, even defects may get at much later stage of the life cycle, So, solution of this problem is : Iterative Waterfall Model.
- Iterative Waterfall Model is by far the most widely used model, almost every other model is derived from the waterfall model.
- The principle of detecting errors as close to its point of introduction as possible – is known as phase containment of errors, phase containment of errors can be achieved by reviewing after every milestone.



Incremental (Evolutionary) Model

- The Incremental Model is a software development approach where the project is divided into smaller, manageable increments or modules, each increment builds on the previous ones, delivering a working piece of the system at regular intervals.
- This model emphasizes partial delivery of functionality early in the development process and continuous refinement through multiple iterations.
- First core module is developed and then new functionalities added in successive versions, each version may be developed using iterative waterfall model, each successive versions is more useful than previous versions.
- After delivery of software changes are minimum in this model.
- The Figure of Incremental Model are depicted below:



- Phases in the Incremental Model in each increment is listed below,
 1. Feasibility Study
 2. Requirements Analysis and Specification
 3. Design
 4. Coding and Unit Testing
 5. Integration and System Testing
 6. Maintenance

- Advantages

- Each successive version more useful work than previous versions.
- Reducing chance of errors.
- More flexible
- User gets a chance to experiment with partially developed software.

- Disadvantages

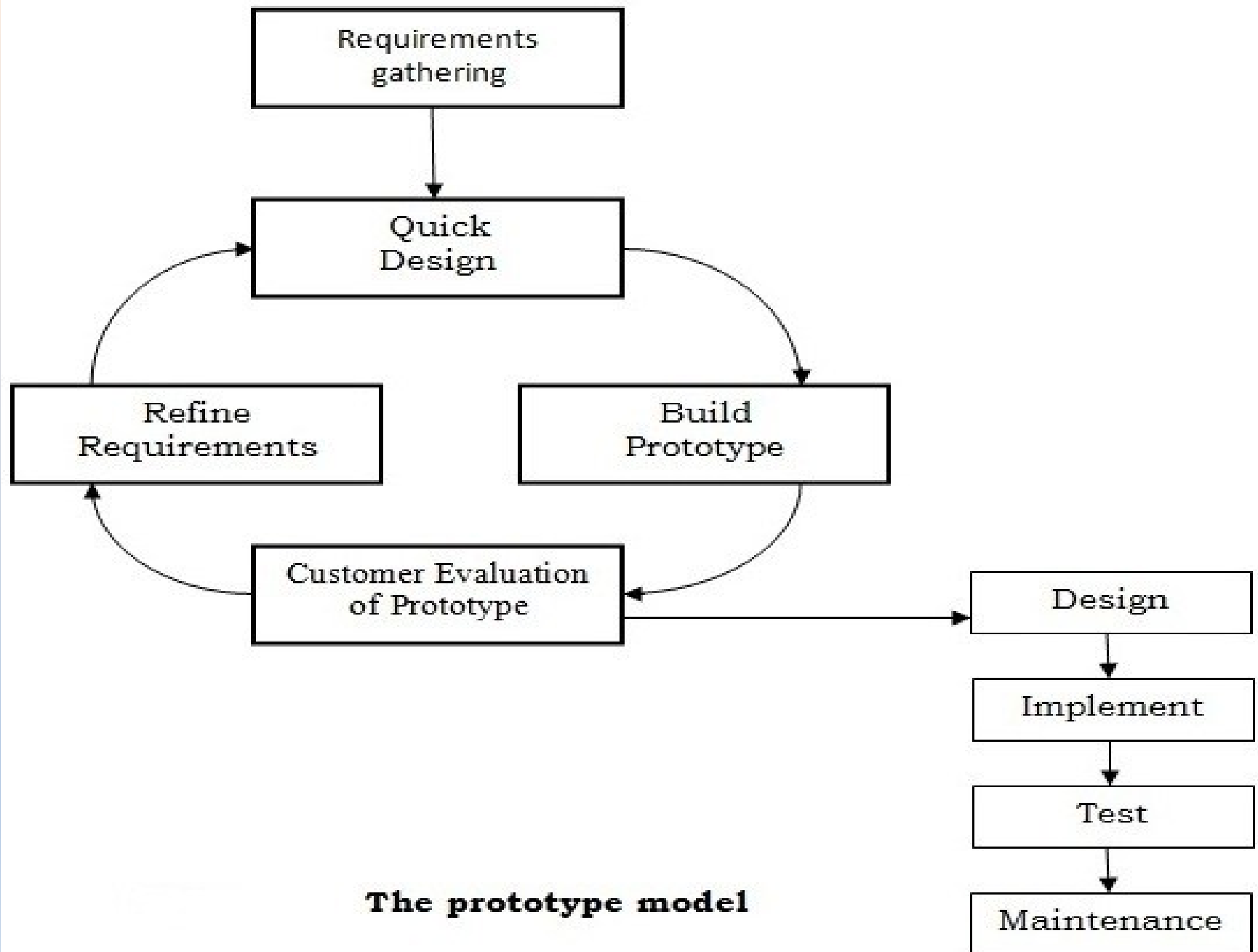
- Sometimes difficult to subdivide the problems into modules.
- Higher Initial Costs
- It needs good planning and design.

- Applications

- When user requirements are not well specified at initial stage.
- Situations where early delivery of critical features is essential.

Prototyping Model

- The Prototyping Model is a software development methodology that focuses on creating prototypes, that is working models of the system with limited functionality to better understand user requirements and validate design concepts.
- In this model, before starting actual development, a working prototype of the system should first be built, a prototype is actually a partial developed product.
- This model is used when user first want to small working model and than actual software or system.
- It is also used when technical solutions are not clear to the development team, In this case developer can make prototype and than using this prototype they can solve the technical issues.
- The Model is shown below,



- In working of the prototype model phases are listed below:
 - Initial Requirements Gathering Phase
 - Quick Design Carried Out
 - Prototype is Built
 - Prototype is then submitted to the customer for Evaluation
 - Based on customer feedback, the requirements are refined and prototype is modified if needed, this cycle of obtaining customer feedback and modifying the prototype continues till the customers approve the prototype.
 - Then Actual system is developed using the different phases of iterative waterfall model.

- Advantages

- Customers can get a chance to have a look of the product.
- New requirements can be accommodated easily.
- Missing functionalities identified quickly.
- It provides better flexibility in design and development.
- More chance of user satisfaction.
- Reduced Risk

- Disadvantages

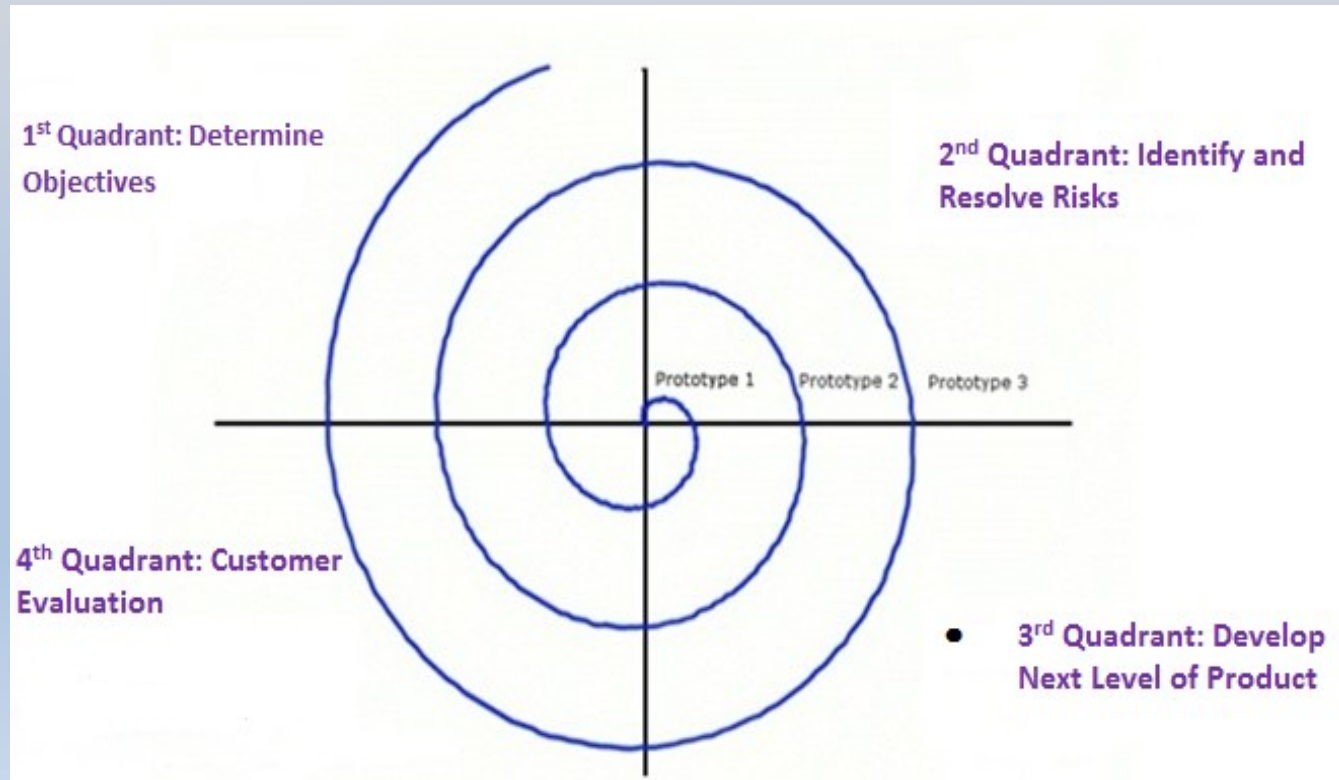
- Required More time
- The construction cost is very high.
- This model requires extensive participation and involvement of the customers that is not possible every time.

- Applications

- When desired system needs to have a lot of interactions with end users.
- Projects with unclear or incomplete requirements.
- Systems requiring high user interaction, such as graphical interfaces or online platforms.

Spiral Model

- This model is proposed by Boehm in 1986.
- The Spiral Model is a risk-driven software development process that combines elements of iterative development with systematic aspects of the Waterfall Model, it emphasizes identifying and addressing risks early in the development process by repeating a set of activities in spirals or loops.
- The diagrammatic representation of this model appears like a spiral with many loops which is shown below,



- Each Loop of the spiral represents a phase of the software process:
 - The innermost loop might be concerned with system feasibility,
 - The next loop with system requirements definition,
 - The next one with system design, and so on.
- No fixed number of phases is there in this model.
- Each loop in the spiral is split into four Sectors (Quadrants)
- **1st Quadrant: Determine Objectives**
 - Identifying the objectives, their relationships and find the possible alternative solutions.
- **2nd Quadrant: Identify and Resolve Risks**
 - Detailed analysis is carried out of each identified risk.
 - Alternate solutions are evaluated and risks are reducing at this quadrant.

- **3rd Quadrant: Develop Next Level of Product**
 - Develop and validate the next level of the product.
 - Activities like design, code development, code inspection, testing and packaging are performed.
 - Resolution of critical operations and technical issues of next level product are performed.
- **4th Quadrant: Customer Evaluation (Review and Planning)**
 - In this part, review the results achieved so far.
 - Plan the next iteration around the spiral different plans like development plan, test plan, installation plan are performed.
- With each iteration around the spiral: progressively more complete version of the software gets built.
- In spiral model, at any point, Radius Represents: cost and Angular dimension represents: progress of the current phase.
- At the end, all risks are resolved and software is ready to use.

- ***Spiral Model Can be Viewed as a Meta Model Because..***
 - Spiral model subsumes almost all the life cycle models.
 - Single loop of spiral represents Waterfall Model.
 - Spiral model uses a prototyping approach by first building the prototype before developing actual product.
 - The iterations along the spiral model can be considered as supporting the Evolutionary Model.

- Advantages

- More Flexible
- Resource Optimization
- Due to user involvement, user satisfaction is improved.
- Risks are analyzed and resolved so final product will be more reliable.
- New idea and additional functionalities can be easily added at later stage.

- Disadvantages

- It is more complex to understand.
- It can be more costly
- More number of documents are needed as a greater number of spirals.

- Applications

- In Large-scale, high-budget, and high-risk projects.
- Systems with frequently changing requirements.
- When flexibility and iterative refinement are desired.
- When requirements are complex.

Agile Methodology

- Agile Methodology is an iterative and incremental approach to software development that emphasizes flexibility, collaboration, customer feedback, and continuous improvement.
- It was introduced to address the limitations of traditional, rigid development methodologies by promoting adaptive planning and evolutionary development.
- Agile Methodology is a step-by-step dynamic focused on short term visibility but never losing the long-term product goal.

Agility Principles (12 Principal of Agile Software)

1. Customer Satisfaction through Early and Continuous Delivery
 - Deliver valuable software early and frequently to meet customer needs and maintain trust.
2. Welcome Changing Requirements
 - Welcome changes in requirements, even late in development, to provide the customer with a competitive advantage.
3. Deliver Working Software Frequently
 - Deliver working software at regular intervals, with a preference for shorter timescales.
4. Collaboration Between Business and Developers
 - Ensure constant and close collaboration between developers and stakeholders throughout the project.
5. Motivate and Support Team Members
 - Build projects around motivated individuals.
6. Face-to-Face Communication
 - Use face-to-face communication as the most effective method of conveying information within a development team.

7. Working Software as a Measure of Progress

- Measure project success by delivering functional, working software, rather than focusing on extensive documentation.

8. Sustainable Development

- Maintain a consistent pace of development that is sustainable for both developers and stakeholders over the long term.

9. Continuous Attention to Technical Excellence

- Focus on technical excellence and good design to enhance agility and deliver high-quality software.

10. Simplicity

- Prioritize simplicity, defined as maximizing the amount of work not done, to avoid unnecessary complexity.

11. Self-Organizing Teams

- Foster self-organizing teams that take ownership of their work, as they are best equipped to design and deliver solutions.

12. Regular Reflection and Adjustment

- At regular intervals, the team reflects on how to become more effective and adjusts its processes accordingly.

Agile Vs Traditionnel Models

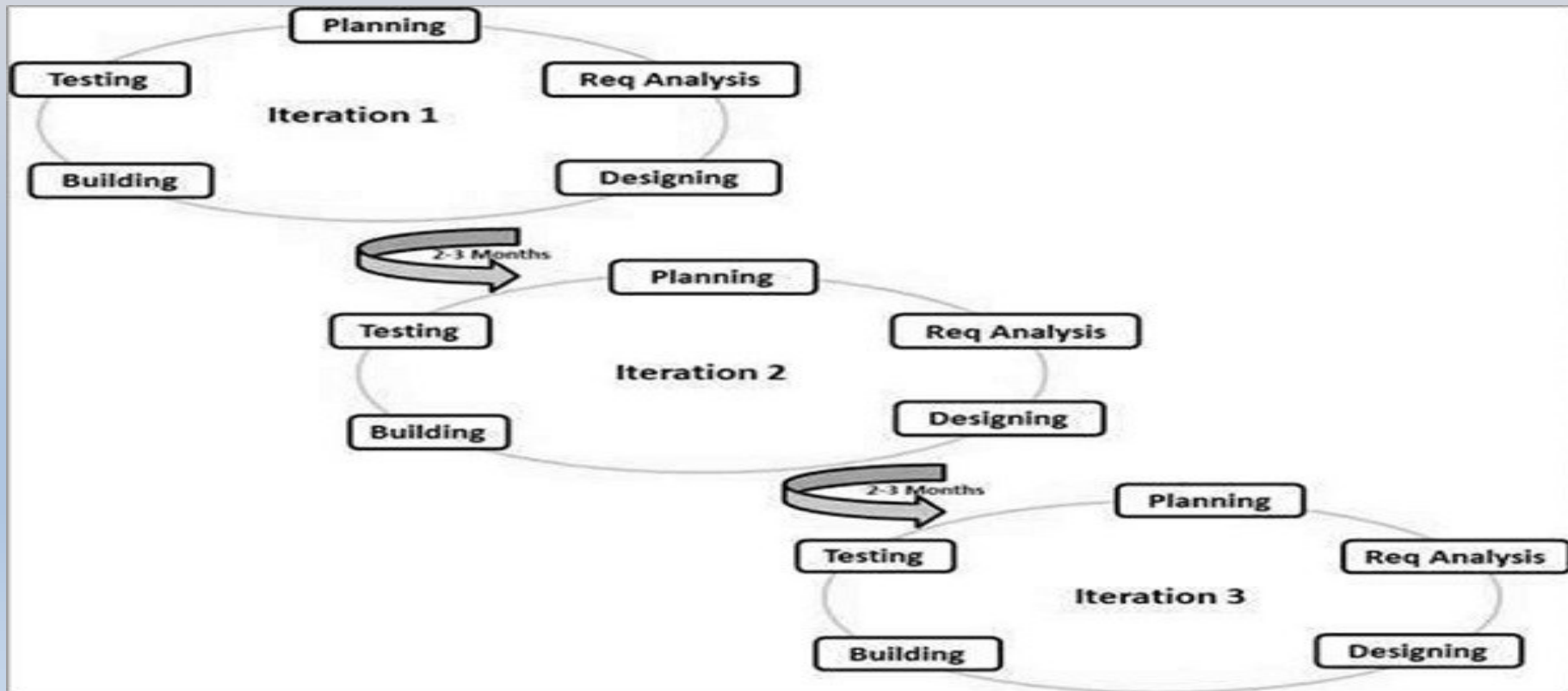
Aspect	Traditional Model	Agile Model
Approach	Linear and Sequential	Iterative and Incremental
Flexibility	Rigid	Flexible
Customer Involvement	Minimal (initial and final stages)	Continuous (throughout the project)
Delivery	Single final delivery	Frequent deliveries of working software
Risk	High (risks addressed early)	Low (continuous risk management)
Project Size	Small, well-defined projects	Complex, evolving projects
Documentation	Extensive	Minimal, focused on essential documentation
Collaboration	Hierarchical, limited collaboration	Collaborative, cross-functional teams
Testing	After development (sequential)	Continuous testing throughout development

Agile Development Model

- Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product.
- Agile Methods break the product into small incremental builds, these builds are provided in iterations.
- Each iteration typically lasts from about one to three weeks.
- Every iteration involves cross functional teams working simultaneously on various areas like:
 - Planning
 - Requirements Analysis
 - Design
 - Coding or Building
 - Testing

- **How It Works?**

- In Agile, the tasks are divided to time boxes (small time frames) to deliver specific features for a release.
- Iterative approach is taken and working software build is delivered after each iteration.
- Each build is incremental in terms of features; the final build holds all the features required by the customer.
- Graphical Representation of Agile Development Model is Depicted Below,



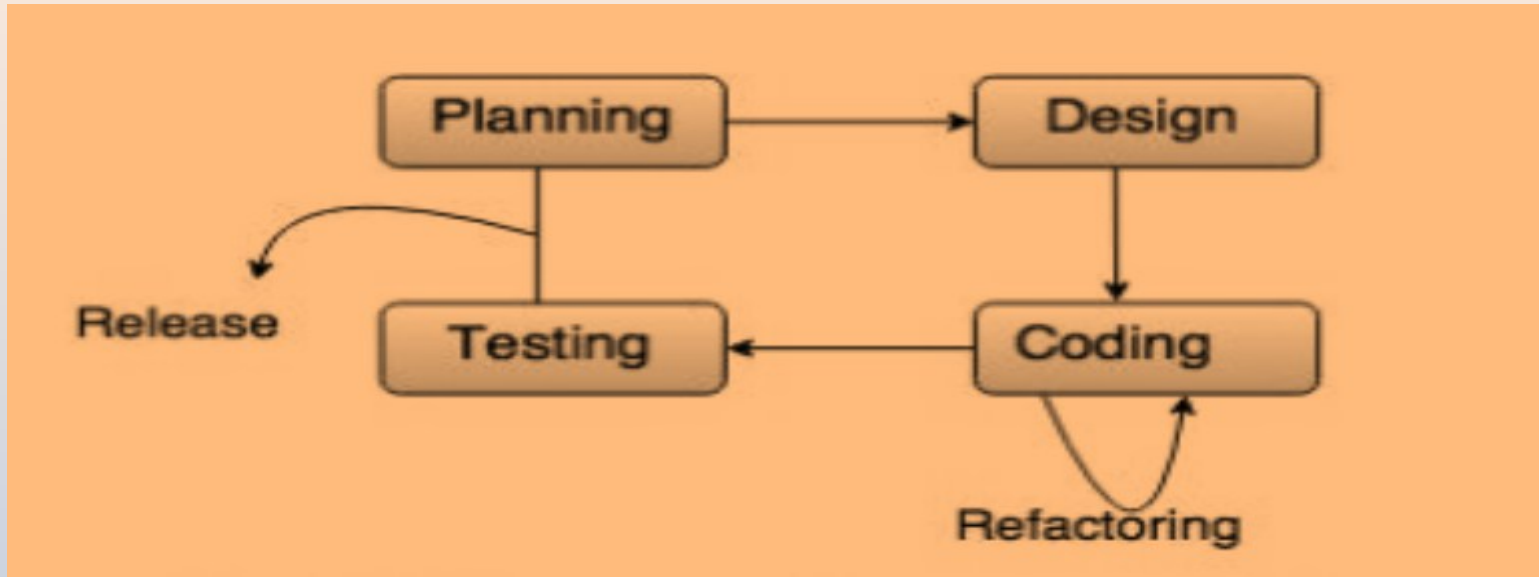
Types of Widely Used Agile Models

- Scrum
- extreme Programming(XP)
- Crystal
- Kanban
- Dynamic Software Development Method(DSDM)
- Feature Driven Development(FDD)
- Adaptive Software Development (ASD)
- Adaptive Project Framework (APF)
- Lean Software Development (LSD)

Extreme Programming(XP)

- This is a Typical Agile Development Framework Developed by Kent Beck.
- This type of Methodology is used when customers are constantly changes demands or requirements or when they are not sure about system performance.
- XP is a lightweight, efficient, low-risk, flexible, predictable, scientific and fun way to develop a software.
- It advocates “Frequent Releases” of the product in short development cycles, which inherently improves the productivity of the system and also introduces a checkpoint where any customer requirements can be easily implemented.
- It Keep customer and quality of the product in target.
- Team work is very important in this model.
- The Sprint (Sprint in Agile project management, is a fixed-length iteration where a defined set of tasks must be completed) is released in very shorter life cycle (almost in 15-20 days).

- The Process of XP Model is Depicted below,



- The Phases are:
 - Planning
 - Design
 - Coding
 - Testing

- Advantages

- Continues Testing and Refactoring Practices Help to Create Stable and well-performing systems with minimal debugging.
- Less number of documents needed.
- Results can be derived soon.

- Disadvantages

- Time consuming as every time customer involvement is necessary.
- Customer involvement is sometime tedious.
- High cost.

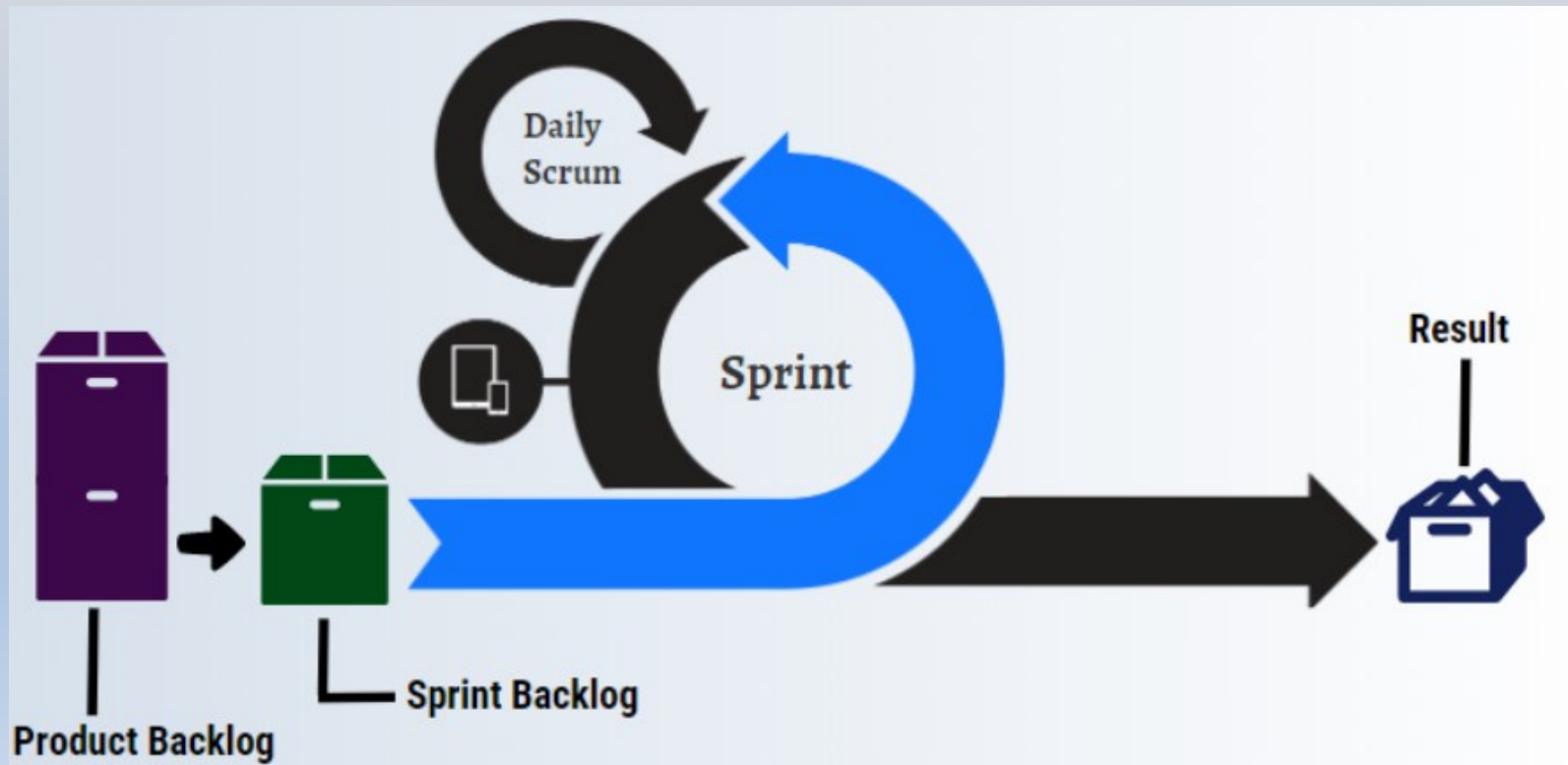
- Applications

- Mainly used for smaller projects.
- Projects involving new technologies and research.

Scrum

- It is mostly used Agile Methodologies.
- It is a light weight agile process framework used for managing software development.
- It focused primarily on ways to manage tasks in team-based development conditions.
- This model based on an iterative and incremental processes.
- It is Adaptable, Fast, Flexible and Effective Agile Framework that is designed to deliver value to the customer throughout the development of the project.
- Scrum is characterized by cycles or stages of development known as “sprints”
- Every day starts with a small 15-minute meeting, called ‘daily scrum’, which takes the role of synchronizing activities and finding the best way to plan out the working day.

- Process of Scrum Model is:
 - A Product Owner Orders the work for a complex problem into a Product Backlog.
 - The Scrum Team turns a selection of the work into an increment of value during a Sprint.
 - The Scrum team and its stockholders inspect the results and adjust for the next Sprint.
 - Repeat the process until the Team get desired Sprint output.



- Three different roles and their responsibilities

1. Scrum Master

- Set up the master team
- Arrange meetings
- Handle clients
- Remove obstacles for the process

2. Product Owner

- What kind of product to be built
- Distribution of functionality on each repetition.

3. Scrum Team

- Complete the cycle of sprint by organizing and managing work.

- Advantages

- It is adaptable and flexible.
- Focus on product quality.
- Improve team motivations.
- More chance of customer satisfaction.

- Disadvantages

- Doesn't work well when more numbers of team members.
- Development team should be well experience.
- Daily meetings sometimes frustrate team members.
- This model is not appropriate for large and complex projects.

- Applications

- Scrum model is a versatile framework that can be applied to a wide range of projects and industries like healthcare, education, marketing, research etc.
- Used when system have rapidly new changes are needed to be implementation required.
- Used when there is a requirement of developing new features in quick time.

GTU Questions

Unit-1 (CO-1)	
Sr No.	Question
1	Define Software Engineering, why there is a need for Software Engineering.
2	Give two examples of each one. i) System Software ii) Artificial Intelligence software.
3	Give two examples of each one. 1) Application Software 2) Embedded software
4	Enlist Software Application Domain and explain AI Software.
5	Enlist and explain Software Design Characteristics of Good Software
6	Explain Software Engineering as a layered Approach with diagram
7	List various Umbrella activities of the software process model. OR Explain Umbrella Activities OR List Various Umbrella activities.
8	Explain Waterfall Model with diagram
9	Explain Iterative Waterfall Model with diagram
10	Write any three advantages and disadvantages of waterfall model.
11	Draw and explain Incremental model, Explain in which situation this model will be used?
12	Write a short note on prototype model
13	Explain spiral model with neat figure.
14	Explain Principles of Agility.
15	Identify changes in the Agile Model approach over traditional model approaches.
16	Differentiate SCRUM Agile process model with SPIRAL process model. Give atleast 4 differences
17	Draw and explain XP Agile process frame work. Explain in which situation this model will be used?
18	List the advantage and disadvantages of Scrum model.
19	Select appropriate Process model for given criteria. i) Clear and fixed user Requirements ii) Medium to high-risk Projects iii) Frequent interaction with end user.